

Computer Graphics: Lecture 16

Lighting part 2: Phong and Gouraud shading

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/)

Welcome back!

Last time we learned about diffuse (Lambertian) shading!

And it was cool, we saw some nice simple shading applied to our models.

But they were a little drab. We couldn't make them shiny.

That changes today...

The teapot will *shine*.

This time

We're going to make several improvements to our lighting code:

- We're going to make it capable of shininess
- We're going to allow lights to have a position in space, so that getting farther away makes them dimmer (attenuation).
- We're going to allow *multiple lights* at one time.
- We're going to improve our shader and the software design of our sample to make bind-group management nicer. This means using more structs in our shader.
- We're going to learn about alignment in WebGPU.
(I couldn't put it off anymore 🙄)

This time (2)

The concepts in this lecture are rather challenging, and in the past I've noticed that we often needed to spend some more time here.

So let's do that. This might be a 2-parter!

Revisiting lambertian lighting

So what's missing from lambertian lighting?

We can clearly see the light on the teapot.

And yet, I've never seen a teapot that looks so...matte before.

Like one of those sports car paintjobs that don't shine at all.



Specular highlights are missing

You know what that sports car doesn't do? Reflect the sun directly into your eyeball when you're eating at a restaurant.

[Has this ever happened to you?]

Those little bright reflections are called specular highlights. They are caused by the photons getting reflected off the surface *right into your eye*. Not getting scattered equally in all directions.

They don't have to be annoying like the sun reflecting off a car. They can give a metallic object its shine. Suffice to say: you're used to seeing them, and not having them is making the mesh look unrealistic!

Specular highlight example



Lambertian-lit teapot.



Teapot with specular highlight
(pay attention to the handle)

How do they work

No one is going to confuse either teapot for the real thing, but clearly we've taken another step toward realism.

Objects without visible specular highlights are curiosities. Even a rock or a tree stump will have some “shine” to it.

But in order to include specular highlights in our shader, we're going to need a more advanced illumination model...

The Phong model

The next model we're going to learn is called **The Phong Reflection** model. It's named after the [Bùi Tường Phong](#)¹, a highly influential computer graphics researcher from the university of Utah. The same place that gave us the teapot.

Bùi sadly passed away very young. However, the reflection model he developed is one of the most visually iconic things from early 3D graphics, and it is still widely used today.

Just to drive this home before we learn it...

¹In Vietnam, the family name comes first, so Phong is actually his given name. It should probably have been named the Bùi model.

The Phong model's importance

The Phong model is found in:

- Natively supported by the Sony Playstation, Nintendo 64, And Atari Jaguar, as well as pretty much every 3D console after that.
- Natively supported by pretty much every 3D modelling software, such as Blender, Maya, and 3DS Max.
- Lots of 80's and 90's era pre-rendered images, such as in [this Stanford competition](#) which goes back to 1992.
- [Movies, too](#)

It gives a distinctive look that accompanies a lot of 90's CG. You'll start to notice it after we learn about it.

From Lambert to Phong

In our Lambertian shader, there were two components to the light:

- Ambient light, which represented the light that was being scattered throughout the scene.
- Diffuse light, which represented the light that was scattered everywhere off the surface.

The Phong reflection model has *three* components:

- Ambient (same as above)
- Diffuse (same as above)
- Specular (new)

These components are added together, for each light, just like before.^{10 / 31}

Specular calculation

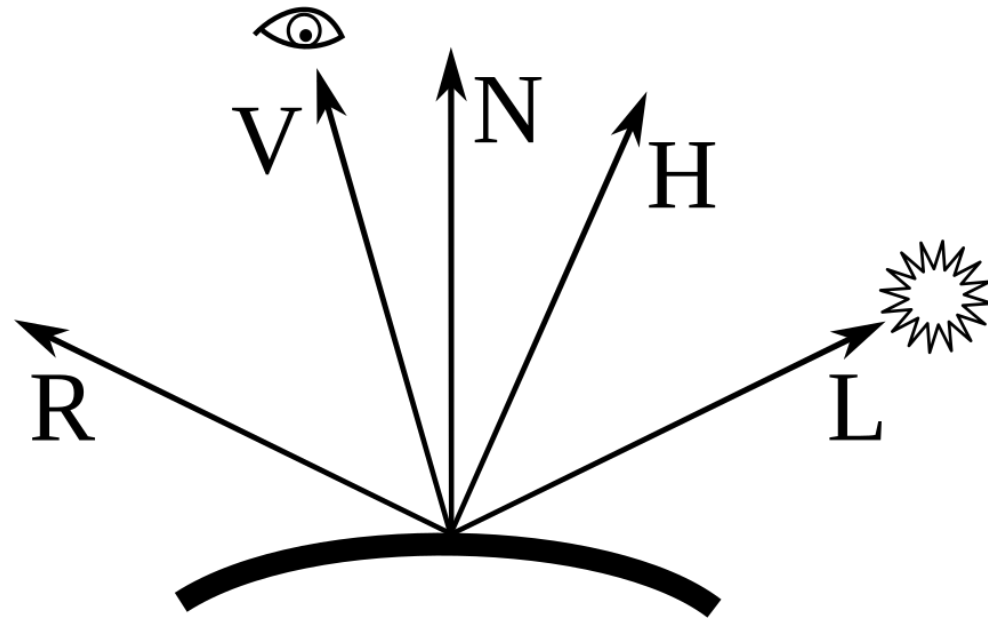
Recall that the diffuse component depended on the cosine between normal of the surface and the direction to the light.

The specular component is a bit more complicated:

- It definitely has to depend on the normal and the direction of the light like before...
- ...However, it represents how light reflects into the eye. So it *also depends on where the eye is*.

Let's see an illustration.

Phong vectors



L is the vector pointing toward the light. N is the normal vector. V is the vector pointing towards the “eye” (camera). R is the reflection of L . [Image](#) by Martin Krause, Public Domain

Phong vectors (2)

Ignore the H vector for now. We'll talk about it later.

The other vectors are all important:

- We need a normal vector like before to tell us how the surface faces.
- This time, the way we use L and N is different. We *reflect* L over N.
- The amount of light that gets reflected is based on the cosine between *the eye and the reflected light*.
- Finally, a *shininess exponent* is applied so falloff is more or less steep.

The result of this calculation is called the **specular component**, but it's only one of three components in the Phong model

The Phong model

The Phong model is a way of calculating light. It's different from **Phong Shading**, which specifically means to use the Phong model in the fragment shader (as opposed to the vertex shader—which is called **Gouraud Shading**).

This is the high level view of the Phong model:

$$\text{ambient color} + \sum_{\text{light}} [\text{diffuse color} + (\text{specular color})^{\text{shininess}}]$$

We've already seen how to compute ambient and diffuse. Specular and shininess are new.

The Phong model (2)

In addition, the colors in the model have two factors:

- The color/brightness¹ of the light
- The color response of the **material**

We briefly saw materials last lecture, where we had ambient and diffuse response factors that represented how the mesh reacted.

Basically, the **material** of your 3D object is where we store its response to different kinds of light. We also often store the texture alongside the material. It describes how the model looks under illumination.

¹We don't really distinguish between the color of the light and its brightness in the Phong model. Brighter lights have higher RGB values.

The Phong material properties

In order to use the Phong model, we have to describe how the object behaves under the 3 different lighting components, as well as its shininess value:

- Ambient: how much ambient light the object reflects
- Diffuse: how much diffuse light the object reflects
- Specular: how much specular light the object reflects
- Shininess: how small and focused the specular highlight is

These can be 4 scalar values, or the first 3 can be RGB values (which I've demonstrated in the sample). Shininess is always a scalar.

The basic model

Since the Phong model is additive, we can think of each kind of brightness as being a separate term. We just add them together:

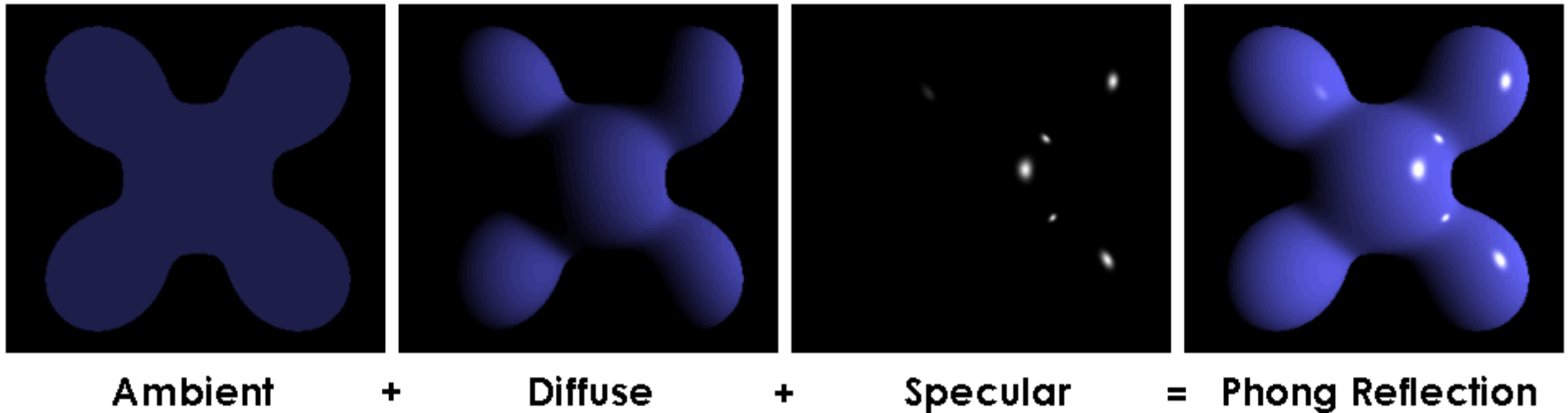


Image by Brad Smith, [CC-BY-SA 3.0](#)

The Phong Equation

The Phong Model is defined by its equation, from which we obtain the color to draw the pixel:

$$I_p = k_a i_a + \sum_{m \in \text{lights}} \left(k_d \left(\hat{L}_m \cdot \hat{N} \right) i_{m,d} + k_s \left(\hat{R}_m \cdot \hat{V} \right)^\alpha i_{m,s} \right)$$

Here are what the variables mean:

- I_p the result we want to compute: either the color of a pixel or its brightness. It will be a brightness if we use scalars for all the lowercase variables, it will be a color if we use 3-vectors and pointwise multiplication.
- The “k” variables are the material properties: ambient, diffuse, and specular respectively.

The Phong Equation (2)

- The “m” subscript describes which light we’re talking about. If there are 8-lights, you will compute their diffuse and specular terms, and add the colors or brightnesses together (the big sigma).
- The $\hat{L}$ is a vector from the surface to the light. $\hat{N}$ is the normal
- $\hat{R}$ is the light vector reflected over the normal.
- Alpha is “shininess”.
- The “i” factors are “illuminance”. Either color or brightness. The subscript tells us which light the factor is for, and which intensity¹

¹I left this out of my sample, because it’s rather abstract, but Phong is an empirical model, and being able to tune the specular “output” of a light may be useful even if it doesn’t make physical sense (specularness is a property of a surface, not a light)

Using the Phong Equation (3)

The way we use this equation is by making all those parameters (except “i” if you want) available to the shader as uniforms.

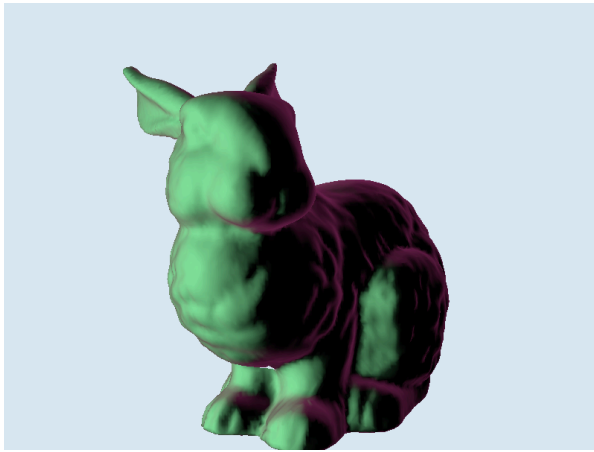
Then we program it into the shader. I’ll show you how I did it in a bit.

Artistically: when we’re building our scene, not only do we decide how to place our meshes and which textures to use, we also give our 3D models material properties. For each model, we decide how much red, green, and blue its specular highlight reflects, how much diffuse and ambient light it reflects, and how small the highlights are. These are all determined in the model’s material, and one material can be shared by many models. It’s often combined with a texture, too.

Questions?

What shininess does

One thing you might be wondering is what shininess does:



Shininess = 1.25
(ignore the pinkish
light, we'll talk about
it later)



Shininess = 15



Shininess = 50

What shininess does (2)

Higher shininess makes the size of the specular spotlight *smaller*.

The result is usually that the surface looks more metallic.

The lights that we're simulating are infinitely small, so as the surface becomes more shiny, it's like you can see that tiny light in the reflection.

Mathmatically, remember that we're taking the dot-product between the reflected light and the eye. These are both unit vectors: the result will always be between -1 and 1 (and we make sure it's at least 0 to avoid "negative light"). Taking that number to a high power will make it need to be very close to 1 (direct reflection) to avoid falling off.

What shininess does (3)

Ironically, this means that increasing shininess means our model is less...well...shiny. Because it's dimmer.

Typically, we can adjust for this by making the highlights more intense, by cranking up the material properties for specular.

One thing that distinguishes Phong from more modern, physically-based illumination models is that it is not actually modelling things like conservation of energy. It is absolutely possible to make the material reflection “reflect” more light than was emitted.¹

¹I didn't do this in the sample, but you can open the page inspector in your browser and force the controls past 1 for the RGB sliders.

Where Phong?

Now, we're ready to start using the Phong illumination model.

However, we are immediately faced with a decision: *where* to start using the Phong illumination model.

You see, there are actually several versions of Phong. Two versions use the same model, but are in different places...

- Using the Phong model in the fragment shader is called, straightforwardly, **Phong Shading**.
- Using the Phong model in the vertex shader is called **Gouraud Shading**.

Gouraud Shading

Named after *another* University of Utah Ph.D student, [Henri Gouraud](#). Gouraud is French, but in English, the closest approximation of his last name is probably “goo-roh” (not “guh-rahd”).

Gouraud shading in when we do (usually) Phong shading at each vertex, and then, linearly interpolate between pixels.

This might be familiar: it’s what the GPU already does for all return values of the fragment shader: it interpolates them over the fragments.

So: we compute the Phong color in the vertex shader and return it as a vertex output. In the fragment shader we use that as our base color (maybe also multiplying it by a texture sample).

Gouraud Shading (2)

Phong shading generates smooth lights, whereas Gouraud shading generates

Blinn-materials

Alignment

Swizzling

Directional vs Point lights

Homework and Reading

I strongly recommend reading [this article](#) on alignment.